

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

Nombre del alumno:

ANGEL GASPAR REYES

8A

TSU EN TECNOLOGIAS DE LA
INFORMACION: DESARROLLO DE
SOFTWARE Y MULTIPLATAFORMA

TEHUACÁN, PUEBLA, 19 de febrero de 2026

Comunicación asíncrona accesible

1. Async JavaScript (JavaScript Asíncrono)

¿Qué es?

Es la forma en que JavaScript maneja tareas que tardan tiempo en responder, como:

- Peticiones a una API
- Cargar datos externos
- Consultar bases de datos
- Esperar respuesta del servidor

En lugar de bloquear la app, JavaScript sigue funcionando mientras espera la respuesta.

Conceptos clave

- Promise
- async / await
- fetch()
- Callbacks

```
JavaScript

async function obtenerUsuarios() {
  try {
    const respuesta = await fetch("https://jsonplaceholder.typicode.com/users");
    const datos = await respuesta.json();
    console.log(datos);
  } catch (error) {
    console.error("Error al obtener usuarios:", error);
  }
}
```

Permite que la interfaz no se congele mientras se cargan los datos.

2. APIs REST

¿Qué es una API REST?

Una API REST es un servicio que permite que una aplicación se comuniquen con un servidor mediante internet para obtener o enviar datos.

Ejemplo:

- App → pide datos al servidor
- Servidor → responde con JSON

Métodos más usados

- GET → Obtener datos
- POST → Enviar datos
- PUT → Actualizar
- DELETE → Eliminar

Ejemplo de consumo de API

```
JavaScript

fetch("https://api.example.com/productos")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error:", error));
```

¿Por qué es importante?

Porque las apps modernas trabajan con:

- Datos externos
- Servicios en la nube
- Bases de datos remotas

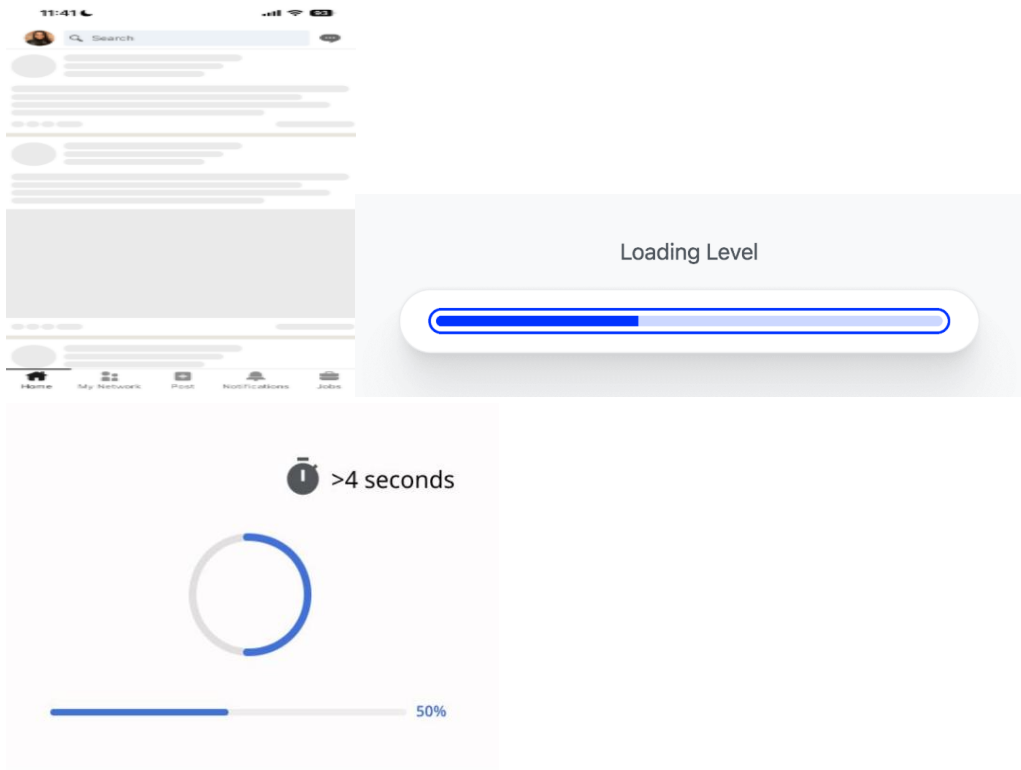
Estados de carga accesibles (Loaders accesibles)

¿Qué son?

Son indicadores visuales o auditivos que informan al usuario que algo se está cargando.

Ejemplos:

- Spinner de carga
- Barra de progreso
- Texto: “Cargando...”



Buenas prácticas de accesibilidad

- Usar texto visible: “Cargando datos...”
- Usar atributos ARIA (aria-live, aria-busy)
- No dejar la pantalla en blanco
- Permitir navegación por teclado

Ejemplo accesible:

```
</> HTML

<div role="status" aria-live="polite">
  Cargando información...
</div>
```

4. Manejo de errores de red

¿Qué son los errores de red?

Ocurren cuando:

- No hay internet
- El servidor falla
- La API no responde
- Timeout (demasiada latencia)

Ejemplo de manejo de errores

```
</> JavaScript

async function cargarDatos() {
  try {
    const res = await fetch("https://api.example.com/data");

    if (!res.ok) {
      throw new Error("Error del servidor");
    }

    const data = await res.json();
    return data;
  } catch (error) {
    console.log("Error de red:", error.message);
  }
}
```

Buenas prácticas UX accesibles

- Mostrar mensaje claro: “Error de conexión”
- Ofrecer botón de reintento
- No solo usar colores (importante para accesibilidad)

¿Para qué sirve todo esto?

Permite que el sistema funcione correctamente incluso cuando:

- Hay latencia (internet lento)
- Fallan servicios externos
- Se usan APIs reales
- Hay errores de conexión

Una app profesional SIEMPRE debe ser tolerante a fallos.

¿Qué debes hacer en tu proyecto? (Interpretación de la tarea)

1. Consumir APIs

Ejemplo:

- API de usuarios
- API de productos
- API externa (JSONPlaceholder, etc.)

2. Implementar loaders accesibles

Ejemplo:

- Texto “Cargando...”
- Spinner con aria-live
- Skeleton screen

3. Manejar errores

Debes documentar:

- Qué pasa si no hay internet

- Qué pasa si falla la API
- Cómo se le informa al usuario

4. Deploy parcial activo

Significa:

- Subir tu proyecto a:
 - Vercel
 - Netlify
 - Render
 - GitHub Pages (si es web)

Entregables explicados (para que no te confundan)

Entregable	Qué significa
Consumo de API funcional	Tu app obtiene datos reales de una API
Estados de carga accesibles	Tiene loader + texto accesible
Manejo de errores documentado	Explicas cómo manejas fallos
Deploy parcial activo	La app está subida y funcionando online